Machine Learning compact cheatsheet

STEP 1 Foundations · Start here

1.1 What machine learning is: learning from examples

Machine learning Learn the rule from examples instead of writing it. Mitchell: performance P on task T improves with experience E.

The loop Guess, measure the loss, adjust parameters against the slope (gradient descent), repeat. Learning rate sets the step size.

Kinds Supervised (labels: regression for numbers, classification for categories), unsupervised (no labels), reinforcement (rewards), plus semi and self supervised.

Words Features are inputs (X), label is the answer (y). Parameters are learned, hyperparameters are chosen.

Generalise Judge a model on unseen test data. High training score with low test score means overfitting.

scikit-learn model = Model(), model.fit(X, y), model.predict(X_new), model.score(X_test, y_test). X is 2D.

A model $\hat{y} = f(x, \theta)$

Mean squared error (loss for numbers)

$$MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$$

One gradient descent step

$$\theta_{new} = \theta - \eta \times (\text{slope of the loss at } \theta)$$

TRAP A learning rate that is too big makes the guess jump past the best value and the loss can grow. One that is too small makes learning very slow. Choosing it well is part of the job.

EXAM A quick test: "Is there a right answer for each example in my data?" Yes means supervised. No means unsupervised. "Does the system learn by acting and getting rewards?" means reinforcement.

EXAM Parameters are learned by the model. Hyperparameters are set by you. The learning rate is a hyperparameter, the slope of a line is a parameter. Interviewers love this question.

TRAP Never measure a model only on its training data. The score will look great and mean nothing. Always keep a separate test set, and look at it only at the end.

TRAP X must be two dimensional: a list of rows, even when there is only one feature. Write [[1], [2], [3]], not [1, 2, 3]. Passing a flat list is the most common first error in scikit-learn.

1.2 The machine learning project: from a question to a working model

Frame first Goal, output type, data known at prediction time, metric, success level, baseline. Ask if ML is needed at all.

Project loop Frame, get data, explore and clean, split, baseline, train and tune, evaluate, deploy, monitor (CRISP-DM plus monitoring).

Three sets Train learns, validation chooses, test scores once. Stratify rare classes, split time by time, keep groups together. Fix random_state.

Baseline DummyRegressor (mean) and DummyClassifier (most frequent) set the bar every model must beat.

Leakage Never train on information unavailable at prediction time. Fit preprocessing on training data only.

Uncertainty SE of an accuracy = √(p(1 − p)/n). Range about p ± 2 SE. Four times the data halves the SE.

Uncertainty of a test accuracy

$$SE = \sqrt{(p(1-p)\ /\ n)}$$

Rough 95% range $p \pm 2 \times SE$

Mean absolute error $MAE = (1/n)\ \Sigma\ |y_i - \hat{y}_i|$

TRAP Look at the data, but do the deep exploration on the training part only, after splitting. If you study the test set too, you start choosing features that fit it, and your final score becomes too optimistic.

EXAM Fix random_state (the seed) so the split is the same every time you run the program. Otherwise your scores change from run to run and you cannot compare models.

TRAP A model that cannot beat the baseline usually means a bug, leakage the other way, or features that carry no signal. Check the data before trying a bigger model.

EXAM If a score looks too good to be true, it usually is. Hunt for leakage before you celebrate.

1.3 Python tools for ML: NumPy, pandas and scikit-learn

NumPy Arrays of one dtype, vectorised maths, masks (a[a > 0]), shape, reshape, .T, np.random.default_rng(seed).

Axis axis=0 goes down rows (one value per column), axis=1 across columns (one per row). The named axis disappears.

Broadcasting Compare shapes from the right. Sizes must be equal or 1. (n, d) minus (d,) centres every column.

pandas DataFrame and Series. df["c"], masks, loc (labels, end included), iloc (positions). groupby with agg or transform, merge, isna, fillna.

scikit-learn fit learns (attributes end in _), transform changes, predict answers. fit_transform on train, transform on test, Pipelines.

Dot product

$$w \cdot x = w_1x_1 + w_2x_2 + ... + w_dx_d$$

Predictions for many rows $\hat{y} = X\ w$

Mean along an axis $mean_j = (1/n)\ \Sigma_i\ X_{ij}$

Standard deviation

$$std = \sqrt{(\ (1/n)\ \Sigma\ (x_i - mean)^2\ )}$$

Standard scaling $z = (x - mean)\ /\ std$

TRAP A NumPy array holds one type. np.array([1, 2.5]) turns everything into floats, and np.array([1, "a"]) turns everything into text. Check .dtype when results look strange.

EXAM axis=0 "collapses" the rows. A tip to remember: the axis you name is the one that disappears from the shape. (4, 3).mean(axis=0) has shape (3,).

EXAM loc uses labels and includes the end: df.loc[0:2] gives 3 rows. iloc uses positions and stops before the end: df.iloc[0:2] gives 2 rows.

TRAP transform keeps one value per original row, so you can put it back into the table as a feature. agg gives one row per group. Mixing them up is a common mistake.

EXAM Call fit_transform on the training set and only transform on the test set. Calling fit on the test set too is a form of data leakage.

STEP 2 Foundations · Maths you need

2.1 Linear algebra for ML: vectors, distances and matrices

Vectors An example is a feature vector. Dot product $a \cdot b = \Sigma\ a_ib_i = ||a||\ ||b||\ \cos\theta$. 0 means orthogonal.

Distances Euclidean $\sqrt{\Sigma(a_i-b_i)^2}$, Manhattan $\Sigma|a_i-b_i|$, cosine $a \cdot b/(||a||\ ||b||)$. Scale features first.

Matrices Matrix times vector: one dot product per row. (n×k)(k×m) = n×m. AB ≠ BA. $(AB)^T = B^TA^T$.

Linear model $\hat{y} = X\ w + b$. Each weight is the change in prediction per unit of its feature.

Inverse det [[a,b],[c,d]] = ad − bc. Inverse exists only if det ≠ 0. Use np.linalg.solve. Dependent columns mean multicollinearity.

Eigen $A\ v = \lambda\ v$. Sum of eigenvalues = trace, product = determinant. Base of PCA.

Euclidean distance (L2)

$$d(a, b) = \sqrt{(\ \Sigma\ (a_i - b_i)^2\ )}$$

Length (norm) of a vector

$$||v||_2 = \sqrt{(v_1^2 + ... + v_d^2)},\ ||v||_1 = \Sigma\ |v_i|$$

Cosine similarity $\cos\theta = a \cdot b\ /\ (||a||\ ||b||)$

Linear model for all examples $\hat{y} = X\ w + b$

2 × 2 determinant

$$\det [[a, b], [c, d]] = a\ d - b\ c$$

Projection $proj_d(x) = ((x \cdot d)\ /\ (d \cdot d))\ d$

Eigen equation $A\ v = \lambda\ v$

Dot product and angle $a \cdot b = ||a||\ ||b||\ \cos\theta$

Squared distance expanded

$$||a - b||^2 = ||a||^2 + ||b||^2 - 2\ a \cdot b$$

TRAP The order of components matters. [age, height] and [height, age] are different vectors. Every example must use the same feature order.

EXAM Distances only make sense when features are on similar scales. Height in centimetres (around 170) would swamp age in years (around 25). Scale features first (Foundations 3.2).

EXAM Matrix multiplication is not commutative: A B is usually not B A, and B A may not even exist when the shapes do not match.

TRAP Perfectly correlated features (one column is a multiple of another) make X^TX impossible to invert. This is called multicollinearity. Drop one of the features or use ridge regression.

2.2 Calculus for ML: derivatives, gradients and gradient descent

Derivative Slope at a point, the limit of (f(x+h) − f(x))/h. Power rule $n\,x^{n-1}$. e^x to e^x, ln x to 1/x, $\sigma' = \sigma(1-\sigma)$.

Chain rule d/dx f(g(x)) = f'(g(x)) g'(x). Slopes along a chain multiply. Base of backpropagation.

Gradient Vector of partial derivatives, points uphill. Update $w \leftarrow w - \eta \nabla L$.

Learning rate Too small is slow, too big diverges. On a x^2, need $\eta < 1/a$. Feature scaling helps.

Variants Batch (all rows), SGD (1 row), mini-batch (32 to 512). Epoch = one pass. Steps per epoch = n / batch size.

Convexity Convex losses (linear and logistic regression) have one minimum. Networks have local minima and saddles.

Gradient
$\nabla f(w) = [\partial f/\partial w_1, \partial f/\partial w_2, ..., \partial f/\partial w_d]$

Gradient descent update $w \leftarrow w - \eta \nabla L(w)$

MSE gradient for a line
$\partial L/\partial w = (2/n) \Sigma (\hat{y}_i - y_i) x_i$, $\partial L/\partial b = (2/n) \Sigma (\hat{y}_i - y_i)$

Convex test (one variable)
$f''(x) \geq 0$ everywhere

Derivative $f'(x) = \lim_{h \to 0} (f(x+h) - f(x)) / h$

Chain rule $d/dx\, f(g(x)) = f'(g(x))\, g'(x)$

Gradient descent $w_{t+1} = w_t - \eta \nabla L(w_t)$

EXAM An epoch is one full pass over the training data. With 1,000 rows and a batch size of 100, one epoch has 10 update steps.

TRAP Gradient descent finds a point where the gradient is near 0. For a convex loss this is the best answer. For a non convex loss it may be a local minimum or a saddle point.

2.3 Probability and statistics for ML

Rules Complement 1 − P(A). Or: P(A) + P(B) − P(A and B). And: P(A) P(B | A). Conditional P(A | B) = P(A and B) / P(B).

Bayes rule P(A | B) = P(B | A) P(A) / P(B), with P(B) from total probability. Rare events: false alarms dominate.

Distributions Bernoulli p, binomial np and np(1 − p), Poisson λ and λ, normal μ and σ^2. 68, 95, 99.7 rule.

Describe Mean, median, Var = $E[X^2] - E[X]^2$, sample std uses n − 1. Correlation in [−1, 1], not causation.

CLT Averages become normal. Standard error σ / √n. 95% interval: estimate ± 1.96 SE.

Likelihood and losses MLE maximises Σ log P(x | θ). Normal noise gives MSE, Bernoulli gives log loss. Gaussian prior gives L2. Cross entropy −Σ p log q.

Bayes rule $P(A \mid B) = P(B \mid A) \times P(A) / P(B)$

Total probability
$P(B) = P(B \mid A)\, P(A) + P(B \mid \text{not } A)\, P(\text{not } A)$

Binomial $P(X = k) = C(n, k)\, p^k\, (1-p)^{n-k}$

Normal density
$f(x) = (1 / (\sigma\sqrt{(2\pi)}))\, e^{-(x-\mu)^2 / (2\sigma^2)}$

Mean (expected value)
$E[X] = \Sigma\, x\, P(x)$, sample mean $\bar{x} = (1/n) \Sigma\, x_i$

Variance
$Var(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2$

Correlation $r = Cov(X, Y) / (\sigma_X\, \sigma_Y)$

Standard error of the mean $SE = \sigma / \sqrt{n}$

Likelihood $L(\theta) = \Pi_i\, P(x_i \mid \theta)$

Log likelihood $\log L(\theta) = \Sigma_i \log P(x_i \mid \theta)$

Coin MLE $\hat{p}$ = heads / n

Entropy $H(p) = -\Sigma\, p_i \log_2 p_i$

Cross entropy $H(p, q) = -\Sigma\, p_i \log q_i$

KL divergence
$KL(p \,||\, q) = \Sigma\, p_i \log(p_i / q_i) = H(p, q) - H(p)$

z score
z = (measured − expected under null) / standard error

95% confidence interval
estimate ± 1.96 × standard error

Conditional probability
$P(A \mid B) = P(A \text{ and } B) / P(B)$

Bayes rule $P(A \mid B) = P(B \mid A)\, P(A) / P(B)$

Expectation and variance
$E[X] = \Sigma\, x\, P(x)$, $Var(X) = E[X^2] - E[X]^2$

Standard error $SE = \sigma / \sqrt{n}$

Log likelihood $\log L(\theta) = \Sigma \log P(x_i \mid \theta)$

Cross entropy $H(p, q) = -\Sigma\, p \log q$

EXAM Ignoring the prior is called the BASE RATE FALLACY . It is a common exam trap and a real problem in fraud and disease detection, where the event you look for is rare.

TRAP The sample standard deviation divides by n − 1 instead of n (ddof=1 in numpy, the default in pandas). This small change corrects for the fact that the sample mean is closer to the data than the true mean is. The program shows 12.9 with n and 13.79 with n − 1.

EXAM Correlation is not causation. Ice cream sales and drownings both rise in summer, but one does not cause the other. Correlation also only measures straight line relations: $y = x^2$ on symmetric x has correlation 0.

EXAM The program shows KL(p || q) = 0.5108 but KL(q || p) = 0.3681. KL divergence is not a true distance because swapping the two distributions changes the answer.

TRAP A p value is not the chance that the null hypothesis is true. If you test 20 useless features at the 0.05 level, about one will look significant by luck. Correct for many tests (for example the Bonferroni correction divides α by the number of tests).

STEP 3

Foundations · Working with data

3.1 Data cleaning: missing values, types, outliers and joins

Inspect shape, dtypes, isna().sum(), duplicated().sum(), unique(), describe(). Write down problems before fixing.

Missing values dropna or fillna. Median for numbers, mode for categories, plus a missing flag. Fit the imputer on training data only.

MCAR, MAR, MNAR Pure chance, depends on other columns, depends on the missing value. Dropping is only unbiased under MCAR.

Types and text str.strip, str.lower, replace, to_numeric and to_datetime with errors="coerce", astype, drop_duplicates.

Outliers IQR fences Q1 − 1.5 IQR and Q3 + 1.5 IQR (robust). |z| > 3 can be masked. Fix, remove, clip or keep.

Combine merge on a key: inner, left, right, outer. Check row counts. melt makes long data, pivot makes wide data.

IQR fences
low = Q1 − 1.5 × IQR, high = Q3 + 1.5 × IQR, IQR = Q3 − Q1

z score $z = (x - \text{mean}) / \text{std}$

Quantile position (pandas linear method)
position = (n − 1) × q, then interpolate between the two nearest sorted values

IQR fences Q1 − 1.5 IQR and Q3 + 1.5 IQR

Mean after mean filling
mean stays the same, variance shrinks by the factor $(n_{known} - 1)/(n - 1)$ or so

TRAP Fitting the imputer on the whole dataset before splitting lets test rows influence the fill value. That is data leakage (Foundations 1.2). Put the imputer inside a Pipeline so it is fitted on the training part only.

EXAM The z score rule missed the 150 in the program: z = 2.97, just under 3. The outlier itself raised the mean and the standard deviation, which hid it. This is called MASKING . The IQR rule uses quartiles, which one extreme value cannot move, so it is more robust.

3.2 Features: encoding, scaling, engineering and selection

Encoding One hot for unordered (handle_unknown="ignore"), ordinal for a real order, target encoding with cross fitting for many categories.

Scaling Standard (x − μ)/σ, min max to 0 to 1, robust (x − median)/IQR, log1p for skew. Trees need none.

No leakage Fit scalers, encoders, imputers and selectors on training data only, inside a Pipeline and ColumnTransformer.

New features Date parts, ratios, bins (pd.cut), logs, group aggregates, sin and cos for cycles, polynomial and interaction terms.

Text CountVectorizer counts words. TF-IDF = tf × idf, idf high for rare words.

Selection Filter (F test, mutual information), wrapper (RFE), embedded (L1, tree importance). Choose k with cross validation.

TF-IDF (scikit-learn smooth version)
tfidf(t, d) = tf(t, d) × (ln((1 + N) / (1 + df(t))) + 1), then each row is scaled to length 1

Standardisation $z = (x - \mu) / \sigma$

Min max $x' = (x - \min) / (\max - \min)$

Robust $x' = (x - \text{median}) / \text{IQR}$

Number of polynomial columns
$C(d + p, p) - 1$

EXAM Trees split on one feature at a time with thresholds like "income > 52". Multiplying a feature by 1,000 moves the threshold but not the split, so tree models do not need scaling.

TRAP Only build features from information you would have at prediction time. "Number of support calls in the month after cancelling" predicts cancelling perfectly, and is useless (and leaky) for a real prediction.

TRAP The number of columns grows fast. With d features and degree p there are C(d + p, p) − 1 columns: 10 features at degree 3 gives 285 columns. Use regularisation (Supervised 1.2) to stop the extra columns from overfitting.

TRAP Selecting features on the whole dataset and then cross validating is leakage: the test folds helped choose the features, so the score is too optimistic. Put the selector inside the Pipeline, as the k loop in the program does.

EXAM Target encoding replaces a category by the average target for that category. Using a row's own target to encode it leaks the answer. scikit-learn's TargetEncoder avoids this with cross fitting: each row is encoded with means from the other folds, and rare categories are pulled towards the overall mean. That is why its numbers in the program differ a little from the plain shop means.

STEP 1

Supervised Learning · Regression

1.1 Linear regression: fit the best straight line

Model and loss $\hat{y} = b_0 + \Sigma b_j x_j$. Minimise $\Sigma(y - \hat{y})^2$. Slope = cov(x, y)/var(x), line through $(\bar{x}, \bar{y})$.

Solving Normal equation $w = (X^T X)^{-1} X^T y$ (use solve), or gradient descent with scaled features.

Metrics MAE, MSE, RMSE, $R^2 = 1 - SSE/SST$, adjusted R^2. Always on test data.

Assumptions Linear, independent errors, equal spread, normal errors for p values. Check residuals for curves and funnels.

Outliers Squared loss is pulled by outliers. Huber or absolute loss is robust. Median minimises absolute error.

Inference statsmodels OLS gives standard errors, p values and confidence intervals. Correlated features make weights unstable.

Simple linear regression
$b_1 = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2$, $b_0 = \bar{y} - b_1\bar{x}$

Least squares loss $L(b) = \Sigma\,(y_i - \hat{y}_i)^2$

Normal equation $w = (X^T X)^{-1} X^T y$

Huber loss
$L_\delta(e) = e^2/2$ if $|e| \le \delta$, otherwise $\delta|e| - \delta^2/2$

Adjusted R^2
$R^2_{adj} = 1 - (1 - R^2)(n - 1)/(n - d - 1)$

Model $\hat{y} = Xw$ (X has a first column of 1s)

Loss $L(w) = \|y - Xw\|^2$

Gradient $\nabla L = -2X^T(y - Xw)$

R^2 $R^2 = 1 - SSE/SST$, $SST = \Sigma(y - \bar{y})^2$

EXAM A feature can be statistically significant and still useless for prediction, or very useful for prediction while its p value is unclear because it is correlated with another feature. Prediction (machine learning) and inference (statistics) answer different questions.

1.2 Polynomial regression and regularisation: ridge, lasso, elastic net

Fit diagnosis Underfit: both errors high (bias). Overfit: train low, test high (variance). Choose complexity on validation data.

Ridge Penalty $\alpha\Sigma w^2$. $w = (X^T X + \alpha I)^{-1} X^T y$. Shrinks, never zeros. Fixes multicollinearity.

Lasso Penalty $\alpha\Sigma|w|$. Soft threshold sets weak weights to 0, so it selects features. Coordinate descent.

Elastic net Mix with l1_ratio. Keeps correlated groups together. Safe default.

Practice Scale first (Pipeline). Tune α with RidgeCV, LassoCV, ElasticNetCV on a log grid. Extend the grid if the best is at the edge.

Theory L1 diamond vs L2 ball. Ridge = Gaussian prior, lasso = Laplace prior. Ridge shrinks along SVD directions by $d^2/(d^2 + \alpha)$.

Soft threshold (one lasso update)
$w_j = \text{sign}(\rho)\,\max(|\rho| - \alpha, 0) / z_j$

Ridge
$\min \|y - Xw\|^2 + \alpha\|w\|_2^2$, $w = (X^T X + \alpha I)^{-1} X^T y$

Lasso $\min (1/2n)\|y - Xw\|^2 + \alpha\|w\|_1$

Elastic net
$\min (1/2n)\|y - Xw\|^2 + \alpha r\|w\|_1 + (\alpha(1 - r)/2)\|w\|_2^2$

EXAM In the program the true model uses 3 of 10 features. At α = 0.5, lasso keeps exactly those 3 and sets the other 7 to 0. Ridge with α = 10 keeps all 10 with small weights.

STEP 2

Supervised Learning · Classification

2.1 Logistic regression: probabilities for classification

Model $P(y = 1 \mid x) = \sigma(w \cdot x + b)$, $\sigma(z) = 1/(1 + e^{-z})$. Score = log odds. Boundary $w \cdot x + b = 0$.

Training Minimise log loss (convex). Gradient $(1/n)\Sigma(p - y)x$. No closed form, use lbfgs or gradient descent.

Threshold Default 0.5. Lower it for higher recall, raise it for higher precision. Choose from costs on validation data.

Multi class Softmax $p_j = e^{z_j}/\Sigma e^z$, loss $-\ln p_{true}$. Or one versus rest.

Interpretation Odds ratio e^w per unit. Scale features for comparable weights.

Penalty C = 1/α, small C = strong L2 penalty. Needed for separable data. Polynomial features give curved boundaries.

Sigmoid $\sigma(z) = 1 / (1 + e^{-z})$

Logistic model $P(y = 1 \mid x) = \sigma(w \cdot x + b)$

Log odds (logit) $\ln(p / (1 - p)) = w \cdot x + b$

Log loss
$L = -(1/n)\,\Sigma\,[y_i \ln p_i + (1 - y_i)\ln(1 - p_i)]$

Gradient $\partial L/\partial w = (1/n)\,\Sigma\,(p_i - y_i)\,x_i$

Softmax $p_j = e^{z_j} / \Sigma_m e^{z_m}$

Model $p = \sigma(w \cdot x + b)$, $\sigma(z) = 1 / (1 + e^{-z})$

Sigmoid derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Log loss and gradient
$L = -(1/n)\Sigma[y \ln p + (1 - y)\ln(1 - p)]$, $\nabla_w L = (1/n)\,X^T(p - y)$

Softmax and its loss
$p_j = e^{z_j} / \Sigma e^{z_m}$, $L = -\ln p_{true}$

TRAP LogisticRegression adds an L2 penalty by default (C = 1.0). If you see a ConvergenceWarning, scale the features or raise max_iter. Do not just ignore it.

EXAM Here 1 means malignant. Moving the threshold from 0.5 down to 0.2 cuts the missed cancers from 2 to 1, and the price is a precision drop from 1.0 to 0.852 (more healthy people sent for extra tests).

TRAP Odds are not probabilities. Tripling the odds from 0.1 to 0.3 moves the probability from 9% to 23%, not to 27%. Near 50% the effect on probability is largest.

EXAM In scikit-learn, C is the inverse of the penalty strength. C = 0.001 shrinks the weights a lot (underfits, 0.891), C = 1 is best here (0.981), and C = 100 hardly penalises (0.963, slightly overfit). Ridge uses α, where a bigger number means a stronger penalty, so do not mix them up.

2.2 k nearest neighbours: learn from similar examples

Algorithm Store data. For a query: distances to all, sort, k nearest, majority vote (or average for numbers). No training, slow prediction O(nd).

Choosing k Small k overfits, large k underfits. k = 1 gives 100% training accuracy. Cross validate, odd k for two classes.

Distance Euclidean, Manhattan, Minkowski p, cosine for text. Scale features first, always.

Variants weights="distance" (1/d). KNeighborsRegressor. NearestNeighbors for similarity search.

High dimensions Distances concentrate, nearest is barely nearer than farthest. Select features or use PCA first.

Speed and theory k-d tree and ball tree for low d, approximate search (HNSW) for embeddings. 1-NN error at most twice Bayes error.

Minkowski distance $d(a, b) = (\Sigma\,|a_i - b_i|^p)^{1/p}$

Cosine similarity $\cos(a, b) = a \cdot b / (\|a\|\,\|b\|)$

Weighted KNN prediction
$\hat{y} = \Sigma\,(1/d_i)\,y_i / \Sigma\,(1/d_i)$ over the k nearest

KNN classification
$\hat{y}$ = most common class among the k nearest $N_k(x)$

KNN regression $\hat{y} = (1/k)\,\Sigma_{i \in N_k(x)}\,y_i$

Euclidean distance $d = \sqrt{\Sigma(a_i - b_i)^2}$

Cost of brute force prediction
$O(n \times d)$ per query

EXAM In the program, k = 1 scores 1.0 on training data but only 0.86 in cross validation. The best is around k = 7 (0.913), and k = 101 underfits (0.82). Training accuracy is useless for choosing k.

TRAP Foundations 3.2 showed scaling raise KNN accuracy on the wine data from 0.691 to 0.949. Scaling is not optional for KNN.

2.3 Naive Bayes: classify by counting clues

Rule $P(c \mid x) \propto P(c)\Pi P(x_j \mid c)$. Features independent given the class. Pick argmax of $\log P(c) + \Sigma \log P(x_j \mid c)$.

Smoothing $P(w \mid c) = (n_{w,c} + \alpha)/(n_c + \alpha V)$. Prevents zeros. Use logs to avoid underflow.

Variants MultinomialNB counts (text), BernoulliNB 0 or 1 (absence counts), GaussianNB numbers (mean, variance), ComplementNB imbalanced text.

Strengths One pass of counting, little data, many features, no scaling. Great text baseline.

Weaknesses Overconfident probabilities, no feature interactions, correlated features double counted. Calibrate if probabilities matter.

Theory Generative. Linear boundary for text. Ng and Jordan: faster to its (higher) best error than logistic regression.

Naive Bayes rule
$$P(c \mid x_1, \ldots, x_d) \propto P(c)\ \Pi_j\ P(x_j \mid c)$$

Prediction
$$\hat{y} = \operatorname{argmax}_c\ [\log P(c) + \Sigma_j \log P(x_j \mid c)]$$

Laplace (add alpha) smoothing
$$P(w \mid c) = (n_{w,c} + \alpha)\ /\ (n_c + \alpha V)$$

Gaussian likelihood
$$p(x_j \mid c) = (1\ /\ \sqrt{(2\pi\sigma_{jc}{}^2)})\ \exp(-(x_j - \mu_{jc})^2\ /\ (2\sigma_{jc}{}^2))$$

Bayes rule $P(c \mid x) = P(x \mid c)\ P(c)\ /\ P(x)$

Naive assumption
$$P(x_1, \ldots, x_d \mid c) = \Pi_j\ P(x_j \mid c)$$

Multinomial likelihood
$$P(w \mid c) = (n_{w,c} + \alpha)\ /\ (n_c + \alpha V)$$

Log score $\text{score}(c) = \log P(c) + \Sigma\ \log P(x_j \mid c)$

EXAM BernoulliNB also uses the words that are missing: a message without "meeting" is a little more likely to be spam. MultinomialNB ignores missing words entirely.

2.4 Support vector machines: the widest street

Margin Boundary $w \cdot x + b = 0$, edges ± 1, width $2/|w|$. Hard margin: $\min \tfrac{1}{2}|w|^2$ with $y(w \cdot x + b) \geq 1$.

Soft margin $\min \tfrac{1}{2}|w|^2 + C\Sigma\max(0, 1 - y\ f(x))$. Small C: wide street, strong regularisation.

Support vectors Points on or inside the margin ($\alpha > 0$). Others can move without changing the model.

Kernels Linear $x \cdot z$, poly $(\gamma x \cdot z + r)^d$, RBF $\exp(-\gamma|x - z|^2)$. Kernel trick: dot products only.

Practice Scale first. Grid search C and γ on a log scale. Big γ overfits. SVC up to about 10^4 to 10^5 rows, else LinearSVC or SGD.

Extras SVR uses an ε tube. No built in probabilities (Platt scaling). Dual solved by SMO in libsvm.

Decision rule $\hat{y} = \text{sign}(w \cdot x + b)$

Margin width $\text{width} = 2\ /\ |w|$

Soft margin objective
$$\min_{w,b}\ \tfrac{1}{2}|w|^2 + C\ \Sigma_i\ \max(0,\ 1 - y_i(w \cdot x_i + b))$$

Epsilon insensitive loss
$$L = \max(0,\ |y - f(x)| - \varepsilon)$$

Hard margin primal
$$\min \tfrac{1}{2}|w|^2 \text{ subject to } y_i(w \cdot x_i + b) \geq 1$$

Soft margin primal
$$\min \tfrac{1}{2}|w|^2 + C\Sigma\xi_i,\ y_i(w \cdot x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

Kernel prediction
$$f(x) = \Sigma_{i \in SV}\ \alpha_i y_i K(x_i,\ x) + b$$

RBF kernel $K(x, z) = \exp(-\gamma|x - z|^2)$

EXAM As C grows the street shrinks from 8.73 to 3.43 and the number of support vectors drops from 162 to 83, because fewer points are allowed inside the street. Test accuracy improves a little and then stops changing. Small C means strong regularisation. Note that C works the opposite way to the ridge penalty λ: C is roughly $1 / \lambda$.

EXAM Scaling alone lifts accuracy from 0.912 to 0.974. The grid search picks C = 100 with γ = 0.001: a wide reach with a high penalty. Both knobs interact, so search them together, never one at a time.

EXAM SVC(probability=True) fits an extra logistic curve to the SVM scores with internal cross validation (Platt scaling). It is slower and its probabilities can disagree with predict. Use decision_function for ranking.

STEP 3

Supervised Learning · Trees and ensembles

3.1 Decision trees: a chain of yes or no questions

Model A chain of yes or no questions. Leaf = majority class (classification) or mean (regression). No scaling needed.

Impurity Gini $1 - \Sigma p^2$, entropy $-\Sigma p \log_2 p$. Split minimises $(n_L/n)I(L) + (n_R/n)I(R)$. IG = drop in entropy.

Overfitting Full trees memorise. Use max_depth, min_samples_leaf, max_leaf_nodes, or ccp_alpha from cost_complexity_pruning_path with CV.

Regression Step function, minimises squared error, cannot extrapolate.

Importance feature_importances_ (impurity, biased to many valued features). permutation_importance on test data is safer.

Weakness High variance, axis parallel cuts, greedy. Fixed by ensembles: forests (3.2) and boosting (3.3).

Gini impurity $G = 1 - \Sigma_k\ p_k{}^2$

Entropy $H = -\Sigma_k\ p_k\ \log_2\ p_k$

Information gain
$$IG = H(\text{parent}) - \Sigma_{\text{child}}\ (n_{\text{child}}/n)\ H(\text{child})$$

Cost complexity $R_\alpha(T) = R(T) + \alpha\ |T|$

Leaf prediction (regression)
$$\hat{y}_{\text{leaf}} = \text{mean of } y_i \text{ in the leaf}$$

Weighted impurity of a split
$$I_{\text{split}} = (n_L/n)\ I(L) + (n_R/n)\ I(R)$$

Cost complexity $R_\alpha(T) = R(T) + \alpha|T|$

EXAM The weather example is the classic one from Quinlan: splitting 14 days by outlook gives an information gain of 0.247 bits, because overcast days are all "play". In practice Gini and entropy choose the same split most of the time, and Gini is a little faster since it needs no logarithm.

EXAM With depth 1 the tree knows only two numbers (0.59 and −0.7), so 0.5, 1.5 and 3.0 all get the same prediction. Depth 5 has 23 steps and follows the sine curve closely, including some of its noise. A tree can never predict outside the range of the training targets, so it cannot extrapolate a trend.

EXAM Five bootstrap samples of the same data give three different root features and between 12 and 19 leaves. Each tree is a reasonable model, but they disagree. That disagreement is variance, and averaging removes it.

3.2 Random forests: many trees vote

Idea Average many trees: vote or mean probability for classes, mean for numbers. Reduces variance, keeps low bias.

Bagging Bootstrap sample of n rows with replacement. About 63.2% distinct rows, 36.8% out of bag (OOB score).

Randomness max_features at each split: sqrt(p) for classification, all for regression. Decorrelates trees.

Variance $\rho\sigma^2 + (1 - \rho)\sigma^2/B$. Lower ρ beats more B.

Settings n_estimators 200 to 500 (more never overfits), max_features, min_samples_leaf, oob_score=True, n_jobs=-1.

Variants BaggingClassifier for any model, ExtraTrees (random cuts, no bootstrap). Compare with a linear baseline.

Chance a row is left out
$$P(\text{out of bag}) = (1 - 1/n)^n\ \to\ 1/e \approx 0.368$$

Variance of an average
$$\text{Var} = \rho\sigma^2 + (1 - \rho)\sigma^2\ /\ B$$

Bagged prediction (regression)
$$\hat{f}(x) = (1/B)\ \Sigma_b\ \hat{f}_b(x)$$

Bagged prediction (classification)
$$\hat{y} = \text{majority vote of } \hat{y}_b, \text{ or argmax of the mean class probabilities}$$

Out of bag share $(1 - 1/n)^n\ \to\ e^{-1} \approx 0.368$

Variance of the forest $\rho\sigma^2 + (1 - \rho)\sigma^2/B$

EXAM This formula is the heart of random forests and appears in The Elements of Statistical Learning, equation 15.1. Lower correlation between trees matters more than more trees.

EXAM With one random feature per split the trees agree least (0.864) but each is weak. With all features (None) the forest is just bagging: the trees agree more (0.919) and test accuracy drops to 0.924. sqrt, here 5 of 30 features, balances the two and scores best (0.953).

3.3 Boosting: small trees that fix each other

Idea Add shallow trees in sequence. Each fits the current mistakes. $F = F_0 + \eta\Sigma h_m$. Reduces bias.

AdaBoost $\alpha = \tfrac{1}{2}\ln((1-\varepsilon)/\varepsilon)$. Wrong points × e^{α}, right × $e^{-\alpha}$. Exponential loss, noise sensitive.

Gradient Fit trees to $-\partial L/\partial F$: $y - F$ (squared), $y - p$ (log loss), $\text{sign}(y - F)$ (absolute).

Tuning Small learning rate (0.05 to 0.1), many trees with early stopping, depth 3 to 6, subsample 0.8, L2 penalty.

Libraries XGBoost (2nd order, $w^* = -G/(H+\lambda)$), LightGBM (histograms, leaf wise), CatBoost (categories), sklearn HistGB.

Versus Forest: parallel deep trees, variance, never overfits with more trees. Boosting: sequential shallow trees, bias, overfits.

Say of a weak learner $\alpha_m = \tfrac{1}{2}\ln((1-\varepsilon_m)/\varepsilon_m)$

Weight update
$w_i \leftarrow w_i\, e^{-\alpha_m y_i h_m(x_i)}$, then normalise

Gradient boosting update
$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$, h_m fitted to $-\partial L/\partial F_{m-1}$

Additive model $F_M(x) = F_0 + \eta \sum_{m=1}^{M} h_m(x)$

Pseudo residuals
$r_{im} = -\partial L(y_i, F)/\partial F$ at $F = F_{m-1}(x_i)$

AdaBoost say $\alpha = \tfrac{1}{2}\ln((1-\varepsilon)/\varepsilon)$

XGBoost leaf weight $w^* = -G/(H+\lambda)$

EXAM The labels + + − − + + − − cannot be split by any single cut. After round 1 the two points it got wrong (x = 5 and 6) carry 0.25 each instead of 0.125, so round 2 picks a stump that gets them right. Three stumps together classify all 8 points. AdaBoost is gradient boosting with the exponential loss $e^{-yF(x)}$.

EXAM With learning rate 1.0 the best test error comes after just 1 tree and then gets much worse (6,489 after 500 trees). With 0.1 the best is after 15 trees. With 0.01 it is after 180 trees, the lowest error of the three (3,180), and still close to it after 500. Smaller steps are slower but safer.

EXAM On this dataset scaled logistic regression is best (0.981) and AdaBoost close behind (0.977). The breast cancer features are mostly linearly related to the label, so a linear model is hard to beat. Boosting shines on larger tables with nonlinear effects and interactions. Always include a simple baseline.

STEP 4

Supervised Learning · Evaluate and improve

4.1 Metrics: measuring a model properly

Confusion TP, FP (false alarm), FN (miss), TN. sklearn rows = true, columns = predicted.

Rates Precision TP/(TP+FP), recall TP/(TP+FN), specificity TN/(TN+FP), F1 = 2PR/(P+R), F_β.

Imbalance Accuracy misleads. Use recall, precision, balanced accuracy, PR AUC. Compare with DummyClassifier.

Threshold Choose from costs with predict_proba. Lower threshold = more recall, less precision.

Curves ROC: TPR vs FPR, AUC = ranking probability. PR curve for rare positives (baseline = positive share).

Regression and probabilities MAE robust, RMSE punishes big errors, R^2 vs mean. Log loss and Brier for probabilities.

Precision $\text{precision} = TP/(TP+FP)$

Recall (sensitivity, TPR)
$\text{recall} = TP/(TP+FN)$

F1 score $F1 = 2PR/(P+R)$

F beta score $F_\beta = (1+\beta^2)\,PR/(\beta^2 P + R)$

False positive rate
$FPR = FP/(FP+TN) = 1 - \text{specificity}$

MAE $MAE = (1/n)\,\Sigma|y_i - \hat{y}_i|$

RMSE $RMSE = \sqrt{(1/n)\,\Sigma(y_i - \hat{y}_i)^2}$

R^2 $R^2 = 1 - SSE/SST$

Log loss
$L = -(1/n)\,\Sigma[y \ln p + (1-y)\ln(1-p)]$

Accuracy $(TP+TN)/(TP+FP+FN+TN)$

Precision, recall
$P = TP/(TP+FP)$, $R = TP/(TP+FN)$

F_β $(1+\beta^2)PR/(\beta^2 P + R)$

AUC as ranking
AUC = P(score of random positive > score of random negative)

EXAM Predicting "never fraud" scores 99.1% accuracy and catches none of the 89 frauds: recall 0 and balanced accuracy 0.5, the same as guessing. Always look at the class you care about.

EXAM AUC equals the share of (positive, negative) pairs where the positive gets the higher score. Here 12 of 16 pairs are ordered correctly, so AUC = 0.75, exactly what scikit-learn reports. This ranking view is the Mann and Whitney U statistic.

EXAM Four predictions are off by 10 and one by 100. MAE is 28, but RMSE is 45.6, pulled up by the single big miss because it is squared. If that miss matters, RMSE tells you. If it is a rare outlier you can live with, MAE is fairer.

4.2 Validation: honest estimates of performance

Splits Train fits, validation chooses, test reports once. Stratify classes. Retrain on train + validation at the end.

Cross validation k folds, each validates once. cross_val_score(model, X, y, cv=5). Report mean ± std. Shuffle sorted rows.

Splitters StratifiedKFold (classes), GroupKFold (groups=), TimeSeriesSplit (time), LeaveOneOut (tiny data).

Leakage Preprocessing on all data, target leaks, group and time leaks, duplicates. Fix: Pipeline, right splitter, drop future features.

Learning curves High bias: curves meet low, get a better model. High variance: gap closing, get more data or regularise.

Selection bias Best of many CV scores is optimistic. Use a locked test set or nested CV.

Cross validation estimate
$CV = (1/k)\,\Sigma_{i=1}^{k}\,\text{score}(\text{model trained without fold i, fold i})$

K fold estimate $CV_k = (1/k)\,\Sigma_i\, E_i$

Standard error of the mean $SE = s/\sqrt{k}$

Fold size n/k rows validate, $n(k-1)/k$ train

Fits for a grid search
$k \times (\text{number of settings}) + 1$ refit

EXAM Three different values of k tie at 0.982 on just 114 validation rows, and the rule "take the first best" picks k = 1, the most fragile choice. A single small validation set is too noisy to choose reliably. Cross validation, next, uses every row for validation once.

EXAM Each patient's label is random, so there is nothing real to learn. With plain KFold the forest scores 0.88 because it recognises the patients from their other rows. GroupKFold reveals the truth: 0.535, close to guessing. This mistake appears in published medical studies.

EXAM The data is pure noise with random labels, so the honest answer is about 0.5. Choosing the 20 best of 5,000 features on all rows, and only then cross validating, reports 0.85. Doing the same selection inside a pipeline gives 0.48. This exact error is the "wrong way" to do cross validation in The Elements of Statistical Learning, section 7.10.2.

4.3 Tuning and calibration

Hyperparameters Chosen before fit (C, gamma, max_depth, learning rate). validation_curve shows under and overfitting.

Search GridSearchCV (all combinations), RandomizedSearchCV (n_iter, loguniform). Keys like svc__C. best_estimator_ is refitted.

Smarter Successive halving and Hyperband, Bayesian optimisation (Optuna). Log scales for C, gamma, alpha, learning rate.

Honest scores best_score_ is optimistic. Nested CV: cross_val_score(GridSearchCV(...), cv=outer). Or a locked test set.

Calibration CalibratedClassifierCV: sigmoid (Platt, small data) or isotonic (more data). calibration_curve, Brier, ECE. Ranking unchanged.

Imbalance class_weight="balanced" ($n/(K\,n_c)$), TunedThresholdClassifierCV for the chosen metric. Judge by recall, precision, PR AUC.

Successive halving
n candidates → n/η → n/η^2 ..., budget × η each round

Platt scaling $p' = 1/(1+\exp(-(A\,s+B)))$

Expected calibration error
$ECE = \Sigma_b\,(n_b/n)\,|\text{mean predicted}_b - \text{observed}_b|$

Grid size
$|\text{grid}| = \Pi_j\,(\text{number of values for hyperparameter j})$

Random search coverage
$P(\text{at least one try in the top q share}) = 1 - (1-q)^n$

Balanced class weight $w_c = n/(K \times n_c)$

Platt scaling $p' = \text{sigmoid}(A\,s+B)$

EXAM The grid tried 12 settings (60 fits) and found 0.979. Random search with the same number of settings over wide log ranges found C = 10.3 and gamma = 0.015, scoring 0.981, values a coarse grid could never hit. After the search, best_estimator_ is already refitted on all the data you passed.

EXAM On 150 rows with 30 settings, the best inner score is 0.953 but the nested estimate is 0.933. Two points of the inner score were the luck of picking the best of 30. With a locked test set you would see the same drop.

EXAM With 5% positives the default logistic regression finds only half of them (recall 0.5). Balanced weights find 92%, but precision drops to 0.28: more catches, many more false alarms. Whether that trade is right depends on the costs, which is why the threshold should be tuned for the metric you care about.

STEP 1

Unsupervised Learning · Clustering

1.1 k-means: grouping points around centres

Algorithm Choose k centres. Repeat: assign each point to the nearest centre, move each centre to its points' mean. Minimises inertia $\Sigma|x - \mu|^2$.

Local minima Always converges, not always to the best. k-means++ starts ($P \propto D^2$) and n_init > 1.

Choosing k Elbow of inertia vs k, silhouette (b − a)/max(a, b) in [−1, 1], plus the purpose of the clusters.

Practice Scale features. KMeans(n_clusters, n_init, random_state). predict for new points. MiniBatchKMeans for big data.

Limits Round, similar sized clusters. Straight boundaries. Outliers pull centres. Switch to DBSCAN, hierarchical or GMM.

Theory Coordinate descent, NP hard globally, hard assignment limit of EM for spherical Gaussians.

k-means objective (inertia) $J = \Sigma_i |x_i - \mu_{c(i)}|^2$

Update step $\mu_k = (1/|C_k|) \Sigma_{i \in C_k} x_i$

Silhouette of a point $s = (b - a) / \max(a, b)$

Objective $J(c, \mu) = \Sigma_i |x_i - \mu_{c(i)}|^2$

Assign step $c(i) = \text{argmin}_k |x_i - \mu_k|^2$

Update step

μ_k = mean of the points with c(i) = k

k-means++ seeding

$P(\text{choose } x) = D(x)^2 / \Sigma D(x')^2$

EXAM The start is deliberately bad: all three centres begin in the bottom left group. k-means still converges in three rounds, but to a poor answer with inertia 40.5, because the two groups on the right share one centre. It found a local minimum, not the best clustering. Good starting centres matter.

EXAM The data was made with 4 groups, but two of them are close together. The elbow is around k = 3 or 4 and the silhouette prefers k = 3 (0.696 against 0.634). Neither is wrong: at a coarse level there are 3 groups. The final choice depends on what the clusters are for.

EXAM On the two moons k-means cuts straight across both shapes (ARI 0.24). On a tight blob next to a wide one, it moves the boundary into the wide blob and steals its edge points (ARI 0.61), because it always splits halfway between centres.

1.2 Hierarchical clustering and DBSCAN

Agglomerative Start with n clusters, merge the closest two n − 1 times. Dendrogram of merge heights, cut at the biggest jump or at the k you need.

Linkages Single min (chains), complete max (compact), average mean, Ward rise in squared error (round, default, Euclidean only).

DBSCAN Core: at least minPts within eps (self included). Clusters grow through core points. Border joins, rest is noise −1. No k, any shape.

Settings minPts about 2 × features. eps at the knee of the sorted k distance plot. Scale first.

HDBSCAN and OPTICS Handle different densities. HDBSCAN needs only min_cluster_size. OPTICS reachability plot valleys.

Costs and theory Agglomerative n^2 memory. Single linkage = MST. DBSCAN about n log n with an index, n^2 worst case.

Ward merge cost

$\Delta(A, B) = (|A||B| / (|A| + |B|)) \, |\mu_A - \mu_B|^2$

Single linkage

$d(A, B) = \min_{a \in A,\, b \in B} d(a, b)$

Complete linkage

$d(A, B) = \max_{a \in A,\, b \in B} d(a, b)$

Ward merge cost

$\Delta(A, B) = |A||B| / (|A| + |B|) \cdot |\mu_A - \mu_B|^2$

eps neighbourhood

$N_\varepsilon(p) = \{\{ q : d(p, q) \le \varepsilon \}\}$

Core point $|N_\varepsilon(p)| \ge \text{minPts}$

EXAM Six points need exactly five merges. The first three merges join close pairs at distance 1 or 3. Then the jumps grow: 4 and finally 8. The big jump from 4 to 8 says the data has two natural groups at the top level, and the gap from 1 to 3 says three groups also make sense.

EXAM Single linkage follows the two moons perfectly (ARI 1.0) because each moon is a chain of close points. On the blobs it fails completely: it leaves 296 points in one cluster and makes two tiny clusters from 4 stray points. Ward does the opposite: worst on the moons, best on the blobs (0.808). There is no best linkage, only a best linkage for the shape of your data.

EXAM Points 0 to 4 form one dense group and points 5 to 8 another, so all nine are core points and make two clusters. Points 9 and 10 have nobody within 0.8, so they are noise. Nobody asked for 2 clusters: the density alone decided it.

EXAM Half the points have their 5th neighbour within 0.079 and 99% within 0.179, then the largest is 0.285. That knee suggests eps around 0.2, which gives exactly 2 clusters and no noise. At 0.1 the moons break into 8 pieces with 16 noise points, at 0.05 almost everything is noise, and from 0.4 the two moons merge into one cluster.

EXAM With one tight, one wide and one medium group, eps 0.3 suits the tight group but shatters the wide one into pieces (8 clusters, 161 noise points). eps 1.5 suits the wide group but merges two groups (2 clusters). eps 1.0 happens to work here, but you only know that because the true labels are known. HDBSCAN finds the 3 groups (ARI 0.952) without any eps, and OPTICS also finds 3 (ARI 0.94).

1.3 Gaussian mixture models and EM

Model $p(x) = \Sigma \, \pi_k \, N(x \mid \mu_k, \Sigma_k)$. Weights add to 1. Each component has its own mean and covariance.

EM E step: r_{ik} by Bayes rule. M step: $N_k = \Sigma \, r$, $\pi = N_k/n$, weighted means and covariances. Likelihood never falls, local maxima.

Practice GaussianMixture(n_components, covariance_type, n_init). predict_proba for soft labels. Scale features.

Choosing BIC = p ln n − 2 ln L (lower is better) for K and covariance_type. AIC penalises less.

Uses Soft clustering, density estimation, anomaly scores with score_samples, sampling, BayesianGaussianMixture for automatic K.

Theory ELBO and Jensen, k-means as the hard spherical limit, collapse to a point prevented by reg_covar, label switching.

Mixture density $p(x) = \Sigma_k \, \pi_k \, N(x \mid \mu_k, \sigma_k^2)$

Responsibility (E step)

$r_{ik} = \pi_k \, N(x_i \mid \mu_k, \sigma_k^2) \, / \, \Sigma_j \, \pi_j \, N(x_i \mid \mu_j, \sigma_j^2)$

Log likelihood

$\ln L = \Sigma_i \ln \Sigma_k \, \pi_k \, N(x_i \mid \mu_k, \Sigma_k)$

E step

$r_{ik} = \pi_k \, N(x_i \mid \mu_k, \Sigma_k) \, / \, \Sigma_j \, \pi_j \, N(x_i \mid \mu_j, \Sigma_j)$

M step: weights and means

$N_k = \Sigma_i \, r_{ik}$, $\pi_k = N_k / n$, $\mu_k = (1/N_k) \, \Sigma_i \, r_{ik} \, x_i$

M step: covariances

$\Sigma_k = (1/N_k) \, \Sigma_i \, r_{ik} \, (x_i - \mu_k)(x_i - \mu_k)^T$

BIC and AIC

$\text{BIC} = p \ln n - 2 \ln L$, $\text{AIC} = 2p - 2 \ln L$

EXAM The first guess puts both means near the middle. Round by round the curves separate, and by round 9 they sit at 154.8 and 176.2, which are exactly the means of the shorter five and the taller five. The log likelihood rises every round, from −39.65 to −34.52, and then stays put. A height of 165 cm sits between the groups, so it gets a probability of 0.575 for the taller group instead of a hard label.

EXAM The two clusters are long and flat, one above the other. k-means splits them left and right (ARI about 0), because that gives the smaller squared distances. A GMM with one start falls into the same trap. With 5 starts it finds the higher log likelihood (−3.787 against −4.045) and the right answer (ARI 0.941). A point halfway between the clusters gets about 50% for each, and a point inside the lower cluster gets 100%.

EXAM The data was made from 4 blobs. BIC falls steeply up to 4 components (4169) and then rises again, so it picks 4. AIC also has its lowest value at 4 here, but its rise after that is much flatter, because its penalty per parameter is smaller.

EXAM make_blobs made 3 blobs, but two of them overlap so much that they look like one long cluster. Of the 8 components allowed, only 2 keep a weight above 1%, and the others shrink to almost nothing. The prior chose K by itself and judged two overlapping blobs to be one group, which shows that automatic K still needs a check.

EXAM k-means is the limit of EM for a GMM whose components all share one round covariance $\sigma^2 I$, with σ shrinking to 0. Then each responsibility becomes 0 or 1, the E step becomes "nearest centre", and the M step becomes "mean of the points".

STEP 2

Unsupervised Learning · Dimensionality reduction

2.1 Principal component analysis (PCA)

Idea Orthogonal directions of decreasing variance. Eigenvectors of the covariance matrix, eigenvalues are their variances.

Steps Centre (and scale if units differ), covariance or SVD, sort by eigenvalue, keep top k, project $Z = X_c V_k$.

Choosing k explained_variance_ratio_, scree elbow, PCA(0.95), or cross validation of the next model.

Facts $\lambda = s^2/(n-1)$. Reconstruction error = sum of dropped eigenvalues. Max variance = min squared distance.

Pitfalls Unscaled mixed units, fitting before the split, ignoring labels, sign flips, only straight directions.

Variants Kernel PCA for curves, randomized and IncrementalPCA for big data, TruncatedSVD for sparse text.

Covariance matrix $C = X_c^T X_c / (n-1)$

Eigen equation $C v = \lambda v$

Projection $Z = X_c V_k$

First component $v_1 = \text{argmax}_{|v|=1} v^T C v$

Solution $C v_1 = \lambda_1 v_1$, variance $= \lambda_1$

Explained variance ratio $\lambda_j / \Sigma_i \lambda_i$

SVD route $X_c = U S V^T$, $\lambda_j = s_j^2/(n-1)$

Reconstruction error
$(1/(n-1)) |X_c - X_c V_k V_k^T|^2 = \Sigma_{j>k} \lambda_j$

EXAM Hours and scores rise together, so almost all the variation lies along one line. The first component keeps 99.97% of the variance, so each student can be described by one number instead of two with almost no loss. The direction (0.16, 0.99) is mostly along the score axis, because scores vary in tens of points and hours only in single hours. The scaling lesson comes back to this.

EXAM Each digit image has 64 pixel values. The first 2 components keep only 28.5% of the variance, which is enough for a rough picture but not for detail. 21 components keep 90% and 29 keep 95%, so more than half of the 64 numbers are close to redundant.

EXAM In the wine data, proline goes up to 1680 while hue stays below 1.71. Unscaled, the first component explains 99.8% of the variance, but almost all of that is proline, which tells you nothing new. After scaling, the first component explains 36.2% and is led by flavanoids and total phenols, a real combination of related features.

EXAM Two circles, one inside the other, cannot be separated by any straight direction, so one PCA component gives 45% accuracy, no better than guessing. Kernel PCA with gamma 2 turns them into one number that separates them perfectly. With gamma 15 the kernel is too narrow and accuracy drops to 71%, so gamma needs tuning.

EXAM A linear autoencoder with a squared error loss and k hidden units learns the same subspace as PCA with k components, though not necessarily the same axes. So PCA is the simplest autoencoder.

2.2 Manifold learning: t-SNE, UMAP, MDS and Isomap

Manifolds Low dimensional surface in high dimensions. PCA flattens, manifold methods unroll. Isomap: k neighbour graph, shortest paths, classical MDS.

MDS Map from a distance table. Classical MDS: $B = -\tfrac{1}{2} J D^{(2)} J$, top eigenvectors. Metric MDS minimises stress.

t-SNE Gaussian $p_{j|i}$ with perplexity 2^H, Student t q_{ij}, minimise KL(P | Q), early exaggeration, init="pca".

Reading maps Trust neighbours. Not sizes, not gaps. Try several perplexities and seeds. No transform. Not for model input.

UMAP Fuzzy neighbour graph, cross entropy, n_neighbors and min_dist, fast, transforms new points, umap-learn package.

Costs Exact t-SNE $O(n^2)$, Barnes-Hut $O(n \log n)$. Floyd and Warshall $O(n^3)$. PCA to 50 first speeds things up.

Stress $\text{stress} = \Sigma_{i<j} (d_{ij} - |z_i - z_j|)^2$

High dimensional affinity
$p_{j|i} = \exp(-|x_i - x_j|^2 / 2\sigma_i^2) / \Sigma_{k \neq i} \exp(-|x_i - x_k|^2 / 2\sigma_i^2)$

Map affinity (Student t)
$q_{ij} = (1 + |y_i - y_j|^2)^{-1} / \Sigma_{k \neq l} (1 + |y_k - y_l|^2)^{-1}$

Perplexity $\text{Perp}(P_i) = 2^{H(P_i)}$

Classical MDS
$B = -\tfrac{1}{2} J D^{(2)} J$, $J = I - (1/n)\, 1\, 1^T$, $Z = V_k \Lambda_k^{1/2}$

Isomap
D_{geo} = shortest paths in the k neighbour graph, then classical MDS on D_{geo}

t-SNE cost $KL(P | Q) = \Sigma_{i \neq j} p_{ij} \log(p_{ij} / q_{ij})$

t-SNE gradient
$\partial C / \partial y_i = 4 \Sigma_j (p_{ij} - q_{ij})(y_i - y_j)(1 + |y_i - y_j|^2)^{-1}$

EXAM The swiss roll is a flat sheet rolled up in 3D. PCA squashes it flat and mixes the layers, so its best axis follows the position along the roll with a rank correlation of only 0.19. Isomap unrolls it almost perfectly (1.0), spectral embedding nearly so (0.998), and LLE reaches 0.916. All three need a sensible number of neighbours.

EXAM The binary search finds the width 0.607 that gives a perplexity of exactly 3. The three nearest points share almost all the probability (0.437, 0.333, 0.222) and the far points get none. A larger perplexity would widen the Gaussian and spread the probability over more neighbours.

EXAM In 10 dimensions the wide cluster is 10 times as spread as the tight one, but in the t-SNE map both have spread 4.3. The far cluster is 10 times farther than the wide one in 10D, but only about twice as far in the map. So the map tells you which points belong together, not how big groups are or how far apart.

STEP 3

Unsupervised Learning · Patterns, anomalies and recommendations

3.1 Anomaly detection

Robust rules Median and MAD score (x − med)/(1.4826 MAD), flag above 3.5. IQR fences Q1 − 1.5 IQR, Q3 + 1.5 IQR. z scores suffer masking.

Isolation Forest Random feature, random cut. Anomalies isolate fast. $s = 2^{-E[h]/c(n)}$, near 1 anomalous. 256 row samples. contamination sets the cut off.

LOF lrd = 1 / mean reachability distance. LOF = mean neighbour lrd / own lrd. Handles varying density. novelty=True for new points.

Other models One class SVM (nu, gamma), elliptic envelope (one robust Gaussian, Mahalanobis), GMM log density. Match the shape of normal.

Evaluation PR AUC, precision at k, recall at a budget. Not accuracy. Threshold from costs. Refit for drift.

Modes Outlier detection on dirty data, novelty detection on clean data, supervised models when labelled anomalies are plentiful.

Robust score
$(x - \text{median}) / (1.4826 \times \text{MAD})$

Isolation score
$s(x, n) = 2^{-E[h(x)] / c(n)}$, $c(n) = 2(\ln(n-1) + 0.5772) - 2(n-1)/n$

Local reachability density
$\text{lrd}(p) = 1 / \text{mean}_{o \in N_k(p)} \text{reach}_k(p, o)$

LOF $\text{LOF}(p) = \text{mean}_{o \in N_k(p)} \text{lrd}(o) / \text{lrd}(p)$

Robust score $(x - \text{median}) / (1.4826\ \text{MAD})$

Mahalanobis distance
$d_M(x) = \sqrt{((x-\mu)^T \Sigma^{-1} (x-\mu))}$

Isolation score $s = 2^{-E[h]/c(n)}$

LOF mean lrd(neighbours) / lrd(p)

EXAM Two typing mistakes (900 and 950 orders on days that usually see about 50) drag the mean to 185 and the standard deviation to 316. Their z scores are only 2.26 and 2.42, so a $|z| > 3$ rule misses both. This is called MASKING . The median (51) and MAD (4.45) barely move, so the robust scores are about 190 and 200 and both days are flagged, as they are by the IQR fences 41.5 to 61.5.

EXAM With 100 values, the far value 95 is isolated after 1.39 random cuts on average, while a normal value needs about 10. Turning path length into a score with $s = 2^{-h/c(n)}$ gives 0.89 for the far value and 0.44 for the normal one. Scores near 1 are anomalies, and scores below 0.5 are normal.

EXAM A point at distance 1.2 from a group of spread 0.2 is very unusual locally, but it is still much closer to the other points than the edges of the loose group are. LOF ranks it as the single most unusual of 401 points, with an LOF of 4.77 against a typical 1.04. Isolation Forest, which looks at global isolation, ranks it only 13th.

EXAM The normal data is two round blobs. The Gaussian mixture, one class SVM and LOF all score almost perfectly, because they can model two groups. The elliptic envelope assumes one ellipse around everything, so the empty space between the blobs looks normal to it and its PR AUC drops to 0.92. Isolation Forest is close behind at 0.91. Match the method to the shape of normal.

EXAM ROC AUC is 0.957, which sounds excellent. But PR AUC is 0.496, and of the top 100 alerts only half are real. With a budget of 500 alerts the team catches 81% of anomalies but 84% of its work is false alarms. PR AUC and precision at k tell that story, and ROC AUC hides it.

3.2 Association rules and market basket analysis

Measures support P(A ∩ B), confidence P(B | A), lift P(A ∩ B)/P(A)P(B). Lift 1 independent, below 1 negative.

Trap High confidence with a very common B can still mean lift below 1. Always check lift.

Apriori Subsets of frequent sets are frequent. Count level by level, join, prune candidates with infrequent subsets.

Rules Each frequent k itemset gives $2^k - 2$ rules. Filter by support, confidence, lift. Leverage and conviction.

Scale FP-Growth prefix tree, no candidates, two scans. PCY and SON for massive data. Pair counts via X^TX.

Caution Many tests give chance patterns. Rules are correlations, not causes.

Support $\text{supp}(A \Rightarrow B) = P(A \cap B)$

Confidence
$\text{conf}(A \Rightarrow B) = P(B \mid A) = \text{supp}(A \cap B) / \text{supp}(A)$

Lift
$\text{lift} = \text{conf}(A \Rightarrow B) / \text{supp}(B) = P(A \cap B) / (P(A)\,P(B))$

Support $\text{supp}(X) = |\{\{t : X \subseteq t\}\}| / n$

Confidence
$\text{conf}(A \Rightarrow B) = \text{supp}(A \cup B) / \text{supp}(A)$

Lift
$\text{lift}(A \Rightarrow B) = \text{supp}(A \cup B) / (\text{supp}(A)\,\text{supp}(B))$

Apriori property
$X \subseteq Y \Rightarrow \text{supp}(X) \geq \text{supp}(Y)$

EXAM Bread is in 7 of 10 baskets, butter in 5, and both in 5. So every basket with butter also has bread. Confidence(bread ⇒ butter) = 0.5 / 0.7 = 0.714, and lift = 0.714 / 0.5 = 1.43: buying bread makes butter 43% more likely than in a random basket.

EXAM 72% of cereal buyers also buy milk, which sounds like a strong rule. But 80% of all shoppers buy milk. Cereal buyers are slightly less likely to buy milk than everyone else, so the lift is 0.9. Confidence alone can mislead whenever the right hand side is very common.

EXAM With a minimum of 3 baskets, 5 of the 6 items are frequent (biscuits is in only 2). The 5 frequent items give 10 candidate pairs, and 4 pairs survive. Joining those pairs gives triples such as bread, butter and jam, but butter and jam appear together only twice, so that triple is skipped without counting. Only bread, butter and milk is counted, and it is frequent.

EXAM The three habits built into the simulation come out on top: bread with butter (lift 1.97), tea with biscuits (1.82) and coffee with sugar (1.79). Each pair appears in both directions with the same lift but different confidence, because lift is symmetric and confidence is not. 38 of the 44 rules with support of at least 5% have lift below 1, so support alone is a poor filter.

3.3 Recommender systems

Data A huge sparse user by item matrix. Explicit ratings or implicit clicks. Missing is unknown, not a dislike.

Content based Item features, taste profile, cosine similarity. Works for new items, but only finds more of the same.

Neighbours Centre on user means, centred cosine (Pearson) on co rated items, top k, $\hat{r} = \bar{r}_u + \Sigma \text{ sim } (r - \bar{r}_v) / \Sigma |\text{sim}|$.

Factorisation $\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$. Loss on known cells plus λ penalty. SGD or ALS. Tune k and λ on held out data.

Evaluation Precision@k, recall@k, NDCG@k, MRR on a time based split. RMSE ignores order.

Practice Cold start fallbacks, shrunk popularity, two stage retrieval and ranking, feedback loops and popularity bias.

Cosine similarity $\cos(a, b) = a \cdot b / (\|a\|\,\|b\|)$

Centred cosine (Pearson)
$\text{sim}(u, v) = \Sigma_i (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v) / \sqrt{(\Sigma_i (r_{ui} - \bar{r}_u)^2 \Sigma_i (r_{vi} - \bar{r}_v)^2)}$

User based prediction
$\hat{r}_{ui} = \bar{r}_u + \Sigma_v \text{sim}(u, v)(r_{vi} - \bar{r}_v) / \Sigma_v |\text{sim}(u, v)|$

Matrix factorisation $\hat{r}_{ui} = \mu + b_u + b_i + p_u \cdot q_i$

Training loss
$\min \Sigma_{(u,i) \text{ known}} (r_{ui} - \hat{r}_{ui})^2 + \lambda(\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)$

NDCG@k
$\text{DCG@k} = \Sigma_{i=1}^{k} \text{rel}_i / \log_2(i + 1)$, $\text{NDCG@k} = \text{DCG@k} / \text{IDCG@k}$

EXAM The person liked one action sci fi film and one romantic sci fi film, so the profile is 1 for sci fi and 0.5 for romance and action. Star Siege shares sci fi and action and scores 0.866. Every other film shares no feature with the profile and scores 0. This shows the main weakness of content based filtering: it only finds more of the same, and it can never suggest a comedy to someone who has not watched one yet.

EXAM User 1 agrees with user 0 (similarity 0.776) and rated the film 1, well below user 1's mean of 2.75. User 2 disagrees with user 0 (similarity −0.653) and loved the film. Both signals point the same way, so the guess for user 0 is 1.81, far below user 0's mean of 3.33. A negative similarity flips the neighbour's opinion. Many systems simply keep only positive similarities.

EXAM Guessing the average gives a test RMSE of 1.068. For the first 10 epochs SGD is no better, because the small random factors need time to grow. By epoch 20 test RMSE falls to 0.689 and by epoch 60 to 0.36, while train RMSE is 0.2. Run the same program with 30 users, 20 films, only 30% of ratings seen and λ = 0.02, and it overfits: train RMSE 0.194 but test RMSE 1.145, worse than the average guess. With sparse data the penalty is not optional.

EXAM Two of the five recommendations (A and C) are relevant, so precision@5 = 0.4, and two of the three relevant items are found, so recall@5 = 0.667. The hits are at positions 1 and 3, giving DCG = $1 + 1/\log_2 4 = 1.5$. The best possible list would put all three relevant items first, with ideal DCG 2.131, so NDCG@5 = 0.704. The first item is a hit, so the reciprocal rank is 1.

STEP 1

Reinforcement Learning · RL basics

1.1 Agents, rewards and returns

Loop State, action, reward, next state. The agent picks actions with a policy π(a | s).

Return $G_t = r_{t+1} + \gamma\, G_{t+1}$. γ in [0, 1]. Constant reward r gives $r / (1 - \gamma)$.

Goal Maximise the expected return. The best policy is π*.

Trade off Explore to learn, exploit to earn. ε greedy is the simplest balance.

Gymnasium reset(seed) gives obs and info. step(a) gives obs, reward, terminated, truncated, info. Truncated is not terminal.

When to use Actions change future data and feedback is a delayed score. Watch for reward hacking and sample cost.

Return $G_t = \Sigma_{k=0}^{\infty} \gamma^k\, r_{t+k+1}$

Recursive form $G_t = r_{t+1} + \gamma\, G_{t+1}$

Policy $\pi(a \mid s) = P(A_t = a \mid S_t = s)$

Return $G_t = \Sigma_{k \geq 0} \gamma^k\, r_{t+k+1} = r_{t+1} + \gamma\, G_{t+1}$

Goal find π that maximises $E_\pi[G_0]$

EXAM The random agent stepped right, left, left, right, right, right, right. Six moves cost −1 each and the treasure gave +10, so the total was 4. Going right three times in a row would have scored 8. The agent did not know that. Learning means finding it out from experience.

EXAM The rewards −1, −1, −1, +10 add up to 7. With γ = 0.9 the return is 4.58, because the treasure three steps away counts as $0.9^3 \times 10 = 7.29$. With γ = 0 only the first reward counts, −1, and the treasure is invisible. Working backwards gives the return from every step at once: 10, 8, 6.2 and 4.58.

EXAM Over 2,000 episodes: always right earns 8, its exact best path. Right 80% of the time earns 5.29, random earns −7.03 and always left −11. A theory check: a fair random walk from cell 2 between 0 and 5 reaches the treasure first with chance 2/5 and takes 2 × 3 = 6 steps on average, so its expected total is $-5 + 0.4 \times 10 + 0.6 \times (-10) = -7$, just as the simulation found.

EXAM The agent leaned right but did not reach cell 6 within 20 steps, so the episode was TRUNCATED , not terminated. Its last state, cell 2, still had a future. A learner that treated this as a true end would wrongly believe cell 2 is worth nothing after this step.

1.2 Multi armed bandits

Problem One state, k arms, unknown means. Maximise total reward or minimise regret $T\mu^* - \Sigma\, \mu_{a_t}$.

Update $Q \leftarrow Q + (r - Q)/N$. Constant α for drifting arms.

ε greedy Random arm with chance ε, else greedy. Simple, but linear regret with fixed ε.

UCB argmax $Q + c\sqrt{(\ln t / N)}$. Untried arms first. Logarithmic regret. Tune c.

Thompson Beta(1 + wins, 1 + losses). Draw from each, pull the best draw. Probability matching.

Context Contextual bandits (LinUCB) choose per context. Bandits beat fixed A/B tests on reward, A/B tests give cleaner answers.

Running average $Q_{n+1} = Q_n + (r_n - Q_n) / n$

Regret $\text{regret}(T) = T\,\mu^* - \Sigma_t\, \mu_{a_t}$

UCB score
$A_t = \text{argmax}_a\, [Q(a) + c\,\sqrt{(\ln t / N(a))}]$

Action value $q^*(a) = E[R_t \mid A_t = a]$

Incremental form $Q \leftarrow Q + (1/N)(r - Q)$

UCB $\text{argmax}_a\, Q(a) + c\,\sqrt{(\ln t / N(a))}$

Beta posterior
Beta(1, 1) + s clicks and f misses ⇒ Beta(1 + s, 1 + f), mean $(1 + s)/(2 + s + f)$

EXAM The running average gives 4, 5, 5, 6, 5, exactly the plain average at each point, without storing the rewards. With a constant step size of 0.5 the estimate forgets the past: after three rewards of 1 and then rewards of 10 it climbs to 8.859 within three pulls. A constant step size suits arms whose payouts drift over time.

EXAM Averaged over 500 experiments of 1,000 pulls, greedy chooses the best arm only 25% of the time and earns 0.497 per pull. ε = 0.1 earns the most, 0.705, picking the best arm 68% of the time. ε = 0.3 explores too much: it knows the best arm but still pulls a random one 30% of the time.

EXAM Over 2,000 pulls, UCB with c = 1 loses 67.3 in regret, about half of ε greedy's 141.2. But c = 2 does worse than ε greedy (167.7) because its bonus keeps forcing pulls of bad arms, and c = 0.5 lands in between (96.2). UCB is better in principle, yet its bonus size still has to fit the noise in the rewards.

EXAM Over 5,000 visitors and 200 repeats, Thompson sampling earned 323 clicks against 305.3 for an A/B test that split the first half of visitors evenly and then showed the winner. Thompson showed the worst design only 354 times on average, against 844 in the A/B test. The best possible was 350. The A/B test gives a cleaner statistical answer, Thompson gives more clicks while learning.

1.3 Markov decision processes and Bellman equations

Markov Next state depends only on the current state. Chain: P with rows adding to 1. $d_{t+k} = d_t P^k$, stationary $\pi = \pi P$.

MDP (S, A, p, R, γ). MDP + fixed policy = MRP.

MRP: $V = R + \gamma PV$, $V = (I - \gamma P)^{-1} R$.

Values $v_\pi(s) = E[G \mid s]$, $q_\pi(s, a) = E[G \mid s, a]$. $v = \Sigma\, \pi\, q$, $q = \Sigma\, p\, (r + \gamma v)$.

Optimality $v^*(s) = \max_a \Sigma\, p\, (r + \gamma v^*)$. Greedy on q^* is optimal. Nonlinear, solved iteratively.

Convergence Bellman operator is a γ contraction, unique fixed point, error shrinks by γ per sweep.

Gridworld Random policy values −14 to −22, optimal values = −distance to exit. Greedy on v_π improves π.

Markov property
$P(S_{t+1} \mid S_t, S_{t-1}, \ldots, S_0) = P(S_{t+1} \mid S_t)$

k steps ahead $d_{t+k} = d_t\, P^k$

Stationary distribution $\pi = \pi\, P,\ \Sigma\, \pi_i = 1$

State value $v_\pi(s) = E_\pi[G_t \mid S_t = s]$

Action value $q_\pi(s, a) = E_\pi[G_t \mid S_t = s, A_t = a]$

Links
$v_\pi(s) = \Sigma_a\, \pi(a|s)\, q_\pi(s, a)$, $q_\pi(s, a) = \Sigma_{s',r}\, p(s', r \mid s, a)[r + \gamma\, v_\pi(s')]$

MDP
(S, A, p, R, γ), $p(s', r \mid s, a) = P(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)$

MRP solution $V = (I - \gamma P)^{-1}\, R$

Bellman expectation
$v_\pi(s) = \Sigma_a\, \pi(a|s)\, \Sigma_{s',r}\, p(s', r \mid s, a)[r + \gamma\, v_\pi(s')]$

Bellman optimality
$v^*(s) = \max_a\, \Sigma_{s',r}\, p(s', r \mid s, a)[r + \gamma\, v^*(s')]$

EXAM Walking the chain for 10,000 days gives shares 0.468, 0.304 and 0.228. Pushing the starting distribution through P 50 times gives 0.462, 0.308 and 0.231, the same as the left eigenvector of P for eigenvalue 1. Two days ahead from rainy, sunny has chance 0.2 × 0.7 + 0.4 × 0.3 + 0.4 × 0.2 = 0.34.

EXAM Solving the linear system gives Work 9.179, Gym 4.198, Wake −0.138 and Couch −2.282. Repeating $V \leftarrow R + \gamma PV$ from all zeros reaches the same numbers: within 0.01 after 20 sweeps and exactly after 100. Check Work by hand: $10 + 0.9 \times (0.4 \times (-2.282) + 0.6 \times 0) = 9.179$.

EXAM These are the values from Sutton and Barto's example 4.1. Next to an exit the random walker still needs 14 moves on average (value −14), and in the far corners 22. Values are expected costs to finish, so they tell you how hopeless the random policy is, not how short the best path is.

STEP 2

Reinforcement Learning · Solving MDPs

2.1 Dynamic programming: policy and value iteration

DP Known model, sweeps of Bellman backups over all states. Stop when max change < θ.

Evaluation $v(s) \leftarrow \Sigma\, \pi(a|s)\, \Sigma\, p\, (r + \gamma v(s'))$. Error shrinks by γ per sweep. In place is fine.

Policy iteration Evaluate, then greedy improve. Improvement theorem. Few iterations, stops when the policy is stable.

Value iteration $v(s) \leftarrow \max_a\, \Sigma\, p\, (r + \gamma v(s'))$, then greedy extraction. Many cheap sweeps.

Examples Gridworld −14 to −22, FrozenLake start 0.542 (slippery, γ 0.99), gambler bets boldly at 25, 50, 75.

Limits GPI is the pattern of all RL. DP needs a model and suffers the curse of dimensionality. LP solves MDPs in polynomial time.

Policy evaluation update
$v_{k+1}(s) = \Sigma_a\, \pi(a|s)\, \Sigma_{s',r}\, p(s', r \mid s, a)[r + \gamma\, v_k(s')]$

Policy evaluation $v_{k+1} = r_\pi + \gamma\, P_\pi\, v_k$

Greedy improvement $\pi'(s) = \text{argmax}_a\, q_\pi(s, a)$

Policy improvement theorem
$q_\pi(s, \pi'(s)) \geq v_\pi(s)$ for all s ⇒ $v_{\pi'}(s) \geq v_\pi(s)$ for all s

Value iteration
$v_{k+1}(s) = \max_a\, \Sigma_{s',r}\, p(s', r \mid s, a)[r + \gamma\, v_k(s')]$

EXAM After sweep 1 every cell is −1. After sweep 2 a cell next to an exit is $0.25 \times (-1) + 0.75 \times (-2) = -1.75$, and others are −2. With γ = 1 it takes 173 sweeps for the largest change to fall below 0.0001, and the final values are the −14 to −22 seen in the last chapter.

EXAM Starting from "always go left", the value of the start rises from 0 to 0.121, 0.52, 0.532 and finally 0.542 after 7 iterations, when the policy stops changing. The optimal first move at the start is left, into the wall. On ice where you slide sideways two times in three, pushing against the wall means you can only slide down or stay put, never into danger. Good policies on slippery ice look odd but are safe.

EXAM Value iteration reaches exactly the same policy and the same start value, 0.542, as policy iteration, but needs 324 sweeps against 7 policy iterations (each of which hides many evaluation sweeps). The start value stays 0 for the first 5 sweeps because the reward is 6 steps away. The best cells are the two on the bottom row just before the goal: 0.863 and 0.742.

2.2 Monte Carlo and temporal difference learning

MC $V(s) \leftarrow V(s) + \alpha(G - V(s))$. Full returns, no model, unbiased, high variance, needs episode ends.

TD(0) $\delta = r + \gamma V(s') - V(s)$, $V(s) \leftarrow V(s) + \alpha\delta$. Bootstraps, online, lower variance, biased.

Compare TD usually faster on Markov problems. Batch MC fits returns, batch TD fits the Markov model (example 6.4).

Step size Constant α tracks change but fluctuates. Robbins Monro: $\Sigma\,\alpha = \infty$, $\Sigma\,\alpha^2 < \infty$.

n step and λ $G_{t:t+n}$ mixes n rewards and a bootstrap. TD(λ) weights $(1 - \lambda)\lambda^{n-1}$. Traces decay by $\gamma\lambda$.

Warning Deadly triad: function approximation + bootstrapping + off policy can diverge.

Monte Carlo update
$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$

TD(0) update
$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

TD error $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$

Return $G_t = R_{t+1} + \gamma G_{t+1}$

TD(0) $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$

n step return
$G_{t:t+n} = \Sigma_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n})$

EXAM The walk starts in C and moves left or right at random. Reaching the right end gives 1, so the true value of a state is its chance of finishing on the right: 1/6 to 5/6. After one episode MC knows only that one walk ended on the right. After 1,000 episodes its averages are within 0.011 of the truth (RMS error). MC is unbiased but needs many episodes because single returns vary a lot.

EXAM TD reaches an error of 0.075 after 100 episodes, against MC's 0.095 in the previous program. With a constant α of 0.1 it then keeps fluctuating around the truth (0.071 after 1,000 episodes) instead of settling, while MC with sample averages kept improving. A constant step size trades exact convergence for the ability to track change.

EXAM Averaged over 100 runs, TD with α = 0.05 reaches an error of 0.035 after 100 episodes, against 0.094 for MC with either step size. TD with a large α of 0.15 is fastest early (0.095 after 10 episodes) but then stalls near 0.07, because each step jumps too far. Step size matters as much as the method.

2.3 SARSA and Q learning

Action values Learn Q(s, a), act with argmax. No model needed. Explore with ε greedy.

SARSA Target r + γQ(s', a'), a' actually taken. On policy, safe under exploration.

Q learning Target r + γ max Q(s', ·). Off policy, learns q*. Cliff: edge path, worse reward while learning.

Expected SARSA Target r + γ Σ π Q. Lower variance. Greedy π gives Q learning.

Double Q Max of noisy estimates is biased up. Select with Q_1, evaluate with Q_2.

Convergence GLIE exploration, Robbins Monro step sizes, every pair visited. Tabular only. Deep versions need more care.

SARSA update
$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

ε greedy
$\pi(a|s) = 1 - \varepsilon + \varepsilon/|A|$ for the greedy a, and $\varepsilon/|A|$ for others

Q learning update
$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

Bias of the max
$E[\max_a \hat{Q}(a)] \geq \max_a E[\hat{Q}(a)]$

Double Q target
$r + \gamma\, Q_2(s', \text{argmax}_a\, Q_1(s', a))$

SARSA $Q(s, a) \mathrel{+}= \alpha[r + \gamma Q(s', a') - Q(s, a)]$

Q learning
$Q(s, a) \mathrel{+}= \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

Expected SARSA
$Q(s, a) \mathrel{+}= \alpha[r + \gamma \Sigma_{a'} \pi(a'|s') Q(s', a') - Q(s, a)]$

EXAM On the cliff walk SARSA earns an average of −24.42 per episode over its last 100 episodes while still exploring with ε = 0.1. Its greedy path climbs to the top row before heading right. The path is longer than needed, but a random step there costs one move, not a fall of −100.

EXAM Q learning finds the truly shortest path, straight along the cliff edge (13 moves, reward −13). Yet while it still explores, its average reward is only −70.01, far worse than SARSA's, because ε greedy steps near the edge keep sending it over the cliff. Q learning values the policy it would follow without exploring, SARSA values the policy it really follows.

EXAM Averaged over 5 runs, the reward per episode while learning (last 100 of 200 episodes) is −49.2 for Q learning, −34.6 for SARSA and −21.1 for expected SARSA. Expected SARSA learns the safe route like SARSA, with steadier updates. Results vary from run to run, so real comparisons average many seeds.

EXAM In state A, going left leads to B, where every action pays −0.1 on average with a lot of noise, so left is a mistake. Plain Q learning still goes left in 86% of episodes at episode 10 and 16% at episode 300, because the max over B's noisy estimates looks positive. Double Q learning goes left only 40% of the time at episode 10 and settles at the best possible 5% (the random ε moves) by episode 50.

STEP 3

Reinforcement Learning · Modern RL

3.1 Function approximation and deep Q networks

Function approximation $\hat{v}(s, w)$ with few weights. Generalises. Objective VE weighted by $\mu(s)$.

Semi gradient TD $w \mathrel{+}= \alpha[r + \gamma\hat{v}(s') - \hat{v}(s)]\nabla\hat{v}(s)$. Target treated as fixed. Linear gradient = features.

Tile coding Several shifted grids, one active tile each. α about 0.5/n. Mountain car: 1944 weights.

DQN Network Q(s, a, θ), experience replay, target network θ^- copied every C steps, Huber loss.

Improvements Double DQN, dueling V + A − mean A, prioritised replay by |δ|, multi step, distributional, noisy nets = Rainbow.

Deadly triad Approximation plus bootstrapping plus off policy can diverge. Baird's counterexample. Fixes: target networks, gradient TD.

Prediction objective
$VE(\mathbf{w}) = \Sigma_s\, \mu(s)\, [v_\pi(s) - \hat{v}(s, \mathbf{w})]^2$

Gradient Monte Carlo
$\mathbf{w} \leftarrow \mathbf{w} + \alpha[G_t - \hat{v}(S_t, \mathbf{w})]\, \nabla\hat{v}(S_t, \mathbf{w})$

Semi gradient TD(0)
$\mathbf{w} \leftarrow \mathbf{w} + \alpha[R + \gamma\hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})]\, \nabla\hat{v}(S, \mathbf{w})$

Linear value $\hat{v}(s, \mathbf{w}) = \mathbf{w}^T\mathbf{x}(s), \quad \nabla\hat{v} = \mathbf{x}(s)$

DQN loss
$L(\theta) = E_{(s, a, r, s') \sim D}[(r + \gamma \max_{a'} Q(s', a', \theta^-) - Q(s, a, \theta))^2]$

Double DQN target
$y = r + \gamma Q(s', \text{argmax}_a\, Q(s', a, \theta), \theta^-)$

Dueling aggregation
$Q(s, a) = V(s) + A(s, a) - (1/|A|)\, \Sigma_{a'}\, A(s, a')$

Importance ratio $\rho = \pi(a|s) / b(a|s)$

Value error
$VE(\mathbf{w}) = \Sigma_s\, \mu(s)[v_\pi(s) - \hat{v}(s, \mathbf{w})]^2$

Semi gradient TD(0)
$\mathbf{w} \mathrel{+}= \alpha\delta\nabla\hat{v}(S, \mathbf{w})$, $\delta = R + \gamma\hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$

Semi gradient SARSA
$\mathbf{w} \mathrel{+}= \alpha[R + \gamma\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})]\nabla\hat{q}(S, A, \mathbf{w})$

DQN target
$y = r + \gamma(1 - \text{done})\, \max_{a'} Q(s', a', \theta^-)$

EXAM With 10 groups of 100 states, 10 weights stand in for 1000 values. After 2000 episodes gradient Monte Carlo reaches a root mean squared error of 0.058, close to the best any staircase can do. Semi gradient TD reaches 0.11 because its target leans on the next group's weight, which is itself only an average. Its steps near the two ends stay closer to 0.

EXAM Mountain car with 8 tilings of 9 by 9 tiles and 3 actions uses 1944 weights. The first episode takes 2111 steps, because every weight starts at 0, which is optimistic when each step costs −1, so the car tries everything. By episodes 51 to 60 it needs about 149 steps on average. It has learned to rock back and forth to build up speed.

EXAM This DQN is written in plain NumPy: 4 inputs, 32 hidden units and 2 outputs, Adam, a memory of 10,000 and a target copy every 100 steps. It balances the pole for about 11 to 23 steps on average in the first 75 episodes, then 58.6, 131.1 and 171.4 in later blocks of 25 episodes. Ten greedy test runs last 134 to 147 steps out of 200. Deep RL is sensitive to settings and seeds, so real work compares several seeds.

EXAM Hessel and colleagues found that prioritised replay and multi step returns mattered most: removing either one hurt Rainbow the most. Removing dueling or Double Q learning hurt the least on average.

EXAM With linear features the size of the weight vector grows from 10.34 to 452.23 in 1000 steps, although every true value is 0. With a table (one value per state) and the same off policy data, it slowly shrinks, from 2.65 to 2.61. The problem is not off policy data alone, but all three parts of the triad together.

3.2 Policy gradients, actor critic and PPO

Policy gradient Learn $\pi(a|s, \theta)$ by gradient ascent on J. Stochastic and continuous policies. Short corridor: best p = 0.59.

REINFORCE $\theta \mathrel{+}= \alpha G_t \nabla \ln \pi$. Softmax: one hot(a) − π. Monte Carlo, high variance.

Baseline Use G − b(s). Unbiased because $\Sigma_a \nabla \pi = 0$. Best simple choice v(s). Gives the advantage.

Actor critic Critic TD error δ replaces G. GAE: $\hat{A}_t = \Sigma (\gamma\lambda)^l \delta_{t+l}$. A2C, A3C.

PPO and TRPO PPO: min(rA, clip(r, 1 − ε, 1 + ε)A), ε = 0.2, several epochs per batch. TRPO: KL trust region.

Continuous control Gaussian policy $(a - \mu)/\sigma^2$. DDPG, TD3 (double critics, delayed actor, smoothing), SAC (entropy).

Softmax policy
$\pi(a|s, \theta) = e^{h(s, a, \theta)} / \Sigma_b e^{h(s, b, \theta)}$

REINFORCE update
$\theta \leftarrow \theta + \alpha\, G_t\, \nabla \ln \pi(A_t|S_t, \theta)$

Softmax log gradient
$\nabla_h \ln \pi(a) = \text{one hot}(a) - \pi$

Policy gradient theorem
$\nabla J(\theta) \propto \Sigma_s \mu(s) \Sigma_a q_\pi(s, a) \nabla \pi(a|s, \theta)$

Score function form
$\nabla J(\theta) = E_\pi[G_t \nabla \ln \pi(A_t|S_t, \theta)]$

With baseline
$\theta \mathrel{+}= \alpha(G_t - b(S_t)) \nabla \ln \pi(A_t|S_t, \theta)$

One step actor critic
$\theta \mathrel{+}= \alpha_\theta\, \delta\, \nabla \ln \pi(A|S, \theta), \quad \delta = R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$

GAE $\hat{A}_t = \Sigma_{l \geq 0} (\gamma\lambda)^l\, \delta_{t+l}$

PPO clipped objective
$L(\theta) = E[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon)\hat{A}_t)]$

TRPO
$\max E[r_t \hat{A}_t]$ subject to $E[KL(\pi_{old} \,||\, \pi_\theta)] \leq \delta$

Gaussian policy gradient
$\nabla_\mu \ln \pi(a|s) = (a - \mu(s)) / \sigma(s)^2$

Objective $J(\theta) = v_{\pi_\theta}(s_0) = E_{\pi_\theta}[G_0]$

Log derivative trick
$\nabla_\theta \pi(a|s, \theta) = \pi(a|s, \theta)\, \nabla_\theta \ln \pi(a|s, \theta)$

REINFORCE with baseline
$\theta \mathrel{+}= \alpha(G_t - b(S_t))\nabla \ln \pi(A_t|S_t)$

Advantage $A(s, a) = q_\pi(s, a) - v_\pi(s)$

EXAM Solving the Bellman equations for every p shows the best policy is p(right) = 0.59, worth −11.66 per episode, while p = 0.5 is worth −12. REINFORCE starts at 0.5 and, averaged over its last 500 episodes, sits at p = 0.593. Single episodes are noisy, so p wanders around the best value.

EXAM For the fixed policy p = 0.3 the true gradient is 8.48. Over 20,000 episodes the plain REINFORCE estimate averages 8.53 with a standard deviation of 39.65, and with the baseline b(s) = v(s) it averages 8.59 with a standard deviation of 25.33. Same direction on average, about 36% less noise per episode.

EXAM On the cliff walk from chapter 2.3, a tabular actor critic improves from an average reward of −47.1 in its first 100 episodes to about −19 in later blocks. Its most likely path takes 19 moves and visits every row, keeping well away from the cliff edge, much like SARSA. Like SARSA it learns about the policy it actually follows, and as its policy sharpens it explores less, so it earns more than SARSA's −24.42 while learning.

EXAM With the clip, average episode length grows from 23.4 to 172.8 in 15 iterations of about 1000 steps, reusing each batch 10 times. The ratios still leave [0.8, 1.2] a little, because the clip only silences samples already outside the range while the others keep pushing. Without the clip this easy, linear problem also learns, but the ratios swing from 0.11 to 1.92 in the first iteration: the new policy moves far from the data, which with neural networks often ends in collapse.

3.3 RL in practice: models, offline data and RLHF

Model based RL Learn Model(s, a) = (s', r), plan with simulated steps. Dyna-Q: 1 real update plus n planning updates.

MCTS and AlphaZero Select, expand, evaluate, back up. PUCT with a policy prior. Self play. MuZero learns the model.

Offline RL Fixed log, no exploring. Unseen actions overestimated by the max. CQL pushes them down. BC, DAgger.

RLHF SFT, then Bradley–Terry reward model $-\ln \sigma(r_w - r_l)$, then PPO on $r - \beta KL$. $\pi^* \propto \pi_{ref} e^{r/\beta}$.

DPO $r = \beta \ln(\pi/\pi_{ref})$ + const. Loss $-\ln \sigma(\beta[\text{log ratio winner} - \text{log ratio loser}])$.

Safety Reward hacking, side effects, safe exploration, distribution shift, sim to real, many seeds.

Dyna-Q planning update
$Q(S, A) \mathrel{+}= \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$, with $(S', R) = \text{Model}(S, A)$

UCT selection
$a = \text{argmax}\ [Q(s, a) + c\ \sqrt{(\ln N(s) / N(s, a))}]$

AlphaZero selection (PUCT)
$a = \text{argmax}\ [Q(s, a) + c\ P(s, a)\ \sqrt{N(s)} / (1 + N(s, a))]$

Behaviour cloning $\max_\theta \Sigma_{(s, a) \text{ in } D} \ln \pi_\theta(a|s)$

Compounding error
cost of cloning ≤ cost of expert + $O(\varepsilon T^2)$, DAgger: $O(\varepsilon T)$

Bradley–Terry model
$P(y_w \succ y_l) = \sigma(r(y_w) - r(y_l))$

Reward model loss
$L(r) = -E[\ln \sigma(r(x, y_w) - r(x, y_l))]$

KL regularised objective
$\max_\pi E[r(x, y)] - \beta\, KL(\pi(\cdot|x) \,||\, \pi_{ref}(\cdot|x))$

Implicit reward
$r(x, y) = \beta \ln(\pi(y|x) / \pi_{ref}(y|x)) + \beta \ln Z(x)$

DPO loss
$L = -E[\ln \sigma(\beta \ln(\pi(y_w)/\pi_{ref}(y_w)) - \beta \ln(\pi(y_l)/\pi_{ref}(y_l)))]$

Constrained MDP
$\max_\pi E[\Sigma \gamma^t r_t]$ subject to $E[\Sigma \gamma^t c_t] \leq d$

Dyna planning
$Q(s, a) \mathrel{+}= \alpha[\hat{r} + \gamma \max Q(\hat{s}', \cdot) - Q(s, a)]$, $(\hat{s}', \hat{r}) = \text{Model}(s, a)$

UCT $Q(s, a) + c\ \sqrt{(\ln N(s) / N(s, a))}$

Bradley–Terry $P(y_w \succ y_l) = \sigma(r_w - r_l)$

KL regularised optimum
$\pi^*(y) = \pi_{ref}(y)\, e^{r(y)/\beta} / Z$

EXAM Averaged over 5 runs, plain Q learning (n = 0) still needs 271.4 steps in episode 5, Dyna-Q with n = 5 needs 35.6 and with n = 50 only 19.4, against a shortest path of 14. Episode 1 is a random walk for every n, because no reward has been seen yet, so its length differs only by chance.

EXAM The log holds 2742 transitions from a cautious policy that takes a random move 30% of the time, covering 79 of the 96 state and action pairs. Behaviour cloning returns −9. Naive offline Q learning returns −51: one step after the start it picks "right", an action the log never tried, whose value stayed at its starting 0, which looks better than every tried action, and it falls straight into the pit. Pushing unseen actions down gives the best possible −7.

EXAM The reward model learns what the labellers prefer, including their mistake: the made up answer seems best to them and gets the highest learned reward, 2.82, though its true quality is −1. With β = 2 the tuned policy still improves true quality from −0.15 to 0.13. With β = 0.1 it puts all its probability on the made up answer and true quality falls to −1. The KL penalty limits how hard the policy can exploit reward model errors.

EXAM On the same toy problem with honest labellers and β = 1, DPO trained directly on the 400 pairs and the RLHF route (fit a Bradley–Terry reward, then take $\pi \propto \pi_{ref} e^r$) give the same policy, with the largest difference in any probability below 0.0001. Expected quality rises from −0.15 to 0.97.

EXAM Henderson and colleagues (2018) showed that the same deep RL algorithm, with the same settings, can give significantly different results with different random seeds, and that implementation details can matter as much as the algorithm. Report results over several seeds.